



Arnaud Malapert et Marie Pelleau
Université Côte d'Azur, CNRS, I3S, France

Résumé

Et si créer des vidéos des Jeux Olympiques devenait un défi d'algorithmique ?
L'activité Flemme Olympique propose une approche ludique de l'algorithmique à travers un problème de partition. Elle permet d'explorer concrètement des stratégies algorithmiques dans un cadre débranché et de développer la pensée logique.

Table des matières

1	Bienvenue aux Flemmes Olympiques !	2
2	Description du problème	2
3	Partition parfaite des entiers de 1 à n	7
4	Algorithme glouton	8
5	Algorithme glouton répété	11
6	Programmation dynamique	13
7	Citius, altius, fortius (plus vite, plus haut, plus fort)	16
8	Problèmes connexes	17
A	Guide pour les animateurs	18
B	Tableau des instances	19
C	Liste des exercices	19
D	Licence et reproductibilité	20

1 Bienvenue aux Flemmes Olympiques !

C'est le premier jour de votre stage dans une société de production audiovisuelle pour couvrir les jeux olympiques. Vous rêvez d'assister à des exploits sportifs, d'interviewer les plus grands athlètes dans des enceintes célèbres !

Après quelques formalités, vous rencontrez enfin votre chef qui vous décrit votre mission : créer le résumé parfait des Jeux Olympiques, en combinant des séquences de différents sports. Certaines sont courtes, d'autres longues, certaines spectaculaires, d'autres plus calmes. . . et votre objectif est de tout organiser pour créer deux séquences de durée identiques pour les génériques de début et de fin d'une émission.

Pourquoi cette précision ? Les diffusions reposent sur des serveurs capricieux et des automatismes stricts. Une vidéo trop longue ou trop courte peut bloquer tout le système, interrompre d'autres diffusions, et faire grincer des dents dans les salles de contrôle.

La flemme ! Vous partez la tête basse. . . adieu les rêves de lumière. . . et en route pour la salle de montage obscure. Au début, vous tentez d'organiser les séquences à la main, en espérant gagner du temps. Mauvaise idée. Les vidéos s'entrechoquent, certaines séquences débordent, d'autres restent à moitié inutilisées. . . et vous commencez à désespérer.

Après plusieurs heures de vains efforts, un homme à l'air vif et intelligent se présente à vous : « Richard Ernest Bellman, maître des algorithmes, appelé à la rescousse ! Je vais vous aider à trouver une partition parfaite lorsque c'est possible, et la plupart du temps dans un temps raisonnable, sauf dans les pires cas. »

Et pour vous guider, l'esprit d'Alan Jay Perlis, pionnier de la programmation et des langages informatiques, est là pour distiller ses conseils et réflexions sur les algorithmes. . . à travers ses fameuses épigrammes [9].

Sa première leçon : simplifiez le problème. Considérez chaque séquence comme un objet réduit à sa seule durée et préparez-vous à appliquer des algorithmes plus ou moins retors selon les situations.

Alan vous avertit cependant que l'efficacité et les gains de temps n'arriveront qu'après un sérieux effort pour comprendre le problème et dompter les algorithmes. Mais une fois ce coup de maître acquis, vous pourrez enfin profiter de votre flemme olympique. . . sans catastrophe vidéo.

Prêts à devenir des champions de la flemme en domptant la puissance des algorithmes ?

Epigramme d'Alan J. Perlis

On croit comprendre en apprenant, on est plus confiant en écrivant, encore plus en enseignant. . . mais on en est vraiment sûr en programmant.

2 Description du problème

Après la simplification, il faut mettre un peu d'ordre dans tout ça : définir clairement le problème et en explorer les propriétés. Car soyons honnêtes : concevoir – ou même comprendre – un algorithme pour un problème qui nous est étranger, c'est un peu comme écouter une conversation dans une langue étrangère.

Que comprendriez-vous, par exemple, aux conseils d'un marin en pleine régates olympique, si vous n'avez jamais mis un pied sur un bateau? « Je ne vais pas trop tendre le cunningham ni forcer sur le hale-bas au près; je vais plutôt jouer sur l'équilibre du bateau en déplaçant mon poids et ajuster finement le cap, vu la petite brise. »

En clair, sans un peu d'expérience de la mer, ces mots ne sont que du vent !

Définition 2.1 – Sac d'entiers

Un *sac d'entiers* est une collection d'entiers naturels dans laquelle un même nombre peut apparaître plusieurs fois.

Par exemple, le sac

$$S = \{1, 2, 2, 5\}$$

contient quatre entiers et le nombre 2 apparaît deux fois.

Plus généralement, un sac S de n entiers naturels peut être noté

$$S = \{s_1, s_2, \dots, s_n\}$$

où chaque s_i est un entier strictement positif.

Définition 2.2 – Partition d'un sac

Une *partition* d'un sac S consiste à séparer les éléments de S en deux sacs non vides S_1 et S_2 . Chaque élément du sac initial doit apparaître dans *exactement un* des deux sacs.

Autrement dit :

- S_1 et S_2 sont non vides ;
- S_1 et S_2 sont disjoints (aucun entier n'apparaît dans les deux sacs) ;
- S_1 et S_2 recouvrent S (tous les éléments de S apparaissent dans S_1 ou S_2).

Exemple – Partition d'un ensemble

Soit un sac $S = \{1, 2, 3, 4, 5\}$.

- Les sacs S_1 et S_2 forment une partition de S .
 - $S_1 = \{1\}$ et $S_2 = \{2, 3, 4, 5\}$
 - $S_1 = \{2, 4\}$ et $S_2 = \{1, 3, 5\}$
- Les sacs S_1 et S_2 ne forment pas une partition de S .
 - $S_1 = \{1, 2, 3, 4, 5\}$ et $S_2 = \emptyset$, car S_2 est vide.
 - $S_1 = \{1, 2, 3\}$ et $S_2 = \{3, 4, 5\}$, car le nombre 3 apparaît dans les deux sacs.
 - $S_1 = \{1, 2\}$ et $S_2 = \{4, 5\}$, car le nombre 3 n'apparaît dans aucun des deux sacs.

Définition 2.3 – Partition parfaite d'un sac pair

Un sac d'entiers S est dit *pair* si la somme de tous ses entiers est un nombre pair.
 Une *partition parfaite* d'un sac pair est une partition (S_1, S_2) telle que la somme des entiers de S_1 est égale à la somme des entiers de S_2 .

Exemple – Partition parfaite d'un sac pair

$S = \{1, 2, 3, 4\}$ est un sac pair.
 — $S_1 = \{1, 3\}$ et $S_2 = \{2, 4\}$ ne forment pas une partition parfaite, car $1 + 3 \neq 2 + 4$.
 — $S_1 = \{1, 4\}$ et $S_2 = \{2, 3\}$ forment une partition parfaite, car $1 + 4 = 2 + 3$.

Exercice 2.1 – Partition de quatre entiers consécutifs (5 min)

Trouver une partition parfaite pour un sac de quatre entiers consécutifs.

Exemple – Partition de quatre entiers consécutifs

La partition pour les entiers de 1 à 4 se généralisent pour les sacs de quatre entiers consécutifs. Ce sac $S = \{n + 1, n + 2, n + 3, n + 4\}$ est pair ($n \geq 0$).
 — $S_1 = \{n + 1, n + 4\}$, de somme $n + 1 + n + 4 = 2n + 5$,
 — et $S_2 = \{n + 2, n + 3\}$, de somme $n + 2 + n + 3 = 2n + 5$,
 — forment une partition parfaite, car leurs sommes sont égales.

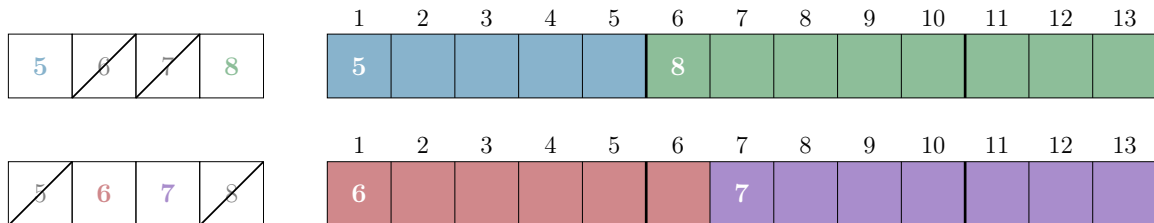


FIGURE 1 – Partition parfaite de quatre entiers consécutifs.

Définition 2.4 – Partition parfaite d'un sac impair

Un sac d'entiers S est dit *impair* si la somme de tous ses entiers est un nombre impair.
 Une *partition parfaite* d'un sac impair est une partition (S_1, S_2) telle que la différence entre la somme des entiers de S_1 et celle de S_2 est égale à 1.
 En effet, lorsque la somme totale est impaire, il est impossible de partager les entiers en deux sacs ayant exactement la même somme. La meilleure partition possible est donc celle où les deux sommes diffèrent de 1.

Exemple – Partition parfaite d'un sac impair

Le sac $S = \{1, 2, 3, 4, 5\}$ est impair car $1 + 2 + 3 + 4 + 5 = 15$.

- $S_1 = \{2, 4\}$ et $S_2 = \{1, 3, 5\}$ ne forment pas une partition parfaite, car $2 + 4 = 6$ et $1 + 3 + 5 = 9$.
- $S_1 = \{1, 2, 5\}$ et $S_2 = \{3, 4\}$ forment une partition parfaite, car $1 + 2 + 5 = 8$ et $3 + 4 = 7$, et la différence entre les deux sommes est 1.

Remarque 2.1

Pour savoir si la somme de tous les entiers d'un sac d'entiers est paire ou impaire, il suffit de compter le nombre d'entiers impairs.

- Si le nombre d'entiers impairs est pair, la somme sera paire.
- Si le nombre d'entiers impairs est impair, la somme sera impaire.

Exercice 2.2 – Partition de six entiers consécutifs (5 min)

Trouver une partition parfaite pour un sac de six entiers consécutifs.

Exemple – Partition de six entiers consécutifs

Le sac $S = \{n + 1, n + 2, n + 3, n + 4, n + 5, n + 6\}$ est impair ($n \geq 0$).

- $S_1 = \{n + 1, n + 4, n + 6\}$, de somme $n + 1 + n + 4 + n + 6 = 3n + 11$,
- et $S_2 = \{n + 2, n + 3, n + 5\}$, de somme $n + 2 + n + 3 + n + 5 = 3n + 10$,
- forment une partition parfaite, car la différence entre les deux sommes est 1.

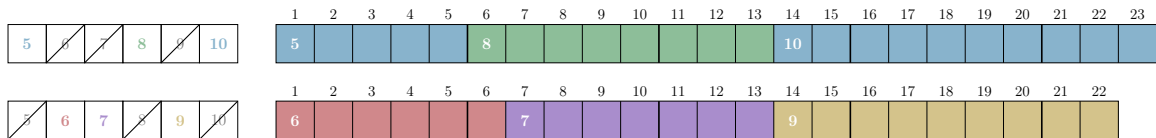


FIGURE 2 – Partition parfaite de six entiers consécutifs.

Définition 2.5 – Problème de partition

En informatique, le problème de partition consiste à déterminer si une partition parfaite d'un sac d'entiers existe.

Ce problème est connu pour être difficile en informatique théorique, classé *NP-complet* :

- on peut vérifier rapidement si une partition candidate est correcte ;
- mais trouver une partition parfaite pour n'importe quel sac peut être très difficile.

Cependant, plusieurs algorithmes permettent de le résoudre de manière efficace pour des instances de taille raisonnable, soit exactement, soit approximativement. Pour cette raison, on l'appelle parfois :

le plus facile des problèmes difficiles.

Remarque 2.2 – Symétries du problème

Ce problème admet une symétrie évidente puisque l'on peut inverser la partition, c'est-à-dire échanger les ensembles S_1 et S_2 .

De manière générale, on peut échanger des nombres entre S_1 et S_2 tant que les sommes des deux sacs restent identiques.

Exemple – Symétries du problème

soit le sac $S = \{1, 2, 4, 6, 8, 9\}$ et la partition parfaite de somme égale à 15.

$$S_1 = \{6, 9\}, \quad S_2 = \{1, 2, 4, 8\}.$$

Inversion de la partition : on peut inverser la partition et les sommes restent identiques.

$$S_1 = \{1, 2, 4, 8\}, \quad S_2 = \{6, 9\}$$

Échange partiel de nombres : on peut aussi échanger des nombres sans changer les sommes, par exemple en remplaçant $6 \in S_1$ par $2, 4 \in S_2$.

$$S_1 = \{2, 4, 9\}, \quad S_2 = \{1, 6, 8\},$$

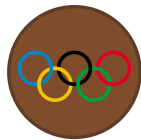
Exercice 2.3 – Échanges partiels (5 min)

Trouver d'autres partitions parfaites de six entiers consécutifs en effectuant des échanges partiels d'entiers entre les deux sacs.

Epigramme d'Alan J. Perlis

La symétrie permet de réduire la complexité ; cherchez-la partout.

3 Partition parfaite des entiers de 1 à n



Avant de se lancer dans des séquences vidéo dignes des Jeux Olympiques, commençons doucement avec des petites séquences faciles à manipuler : les entiers de 1 à n . Explorez les partitions, essayez, échouez, recommencez. Votre mission est de répartir ces nombres en deux résumés parfaits et découvrir que comprendre un problème, c'est souvent plus important que le résultat immédiat.

Exercice 3.1 – Partition parfaite de 1 à n (10 min)

Trouver une partition parfaite des entiers de 1 à n pour $n = 7, 8, 9$.

Remarque 3.1 – Somme des entiers de 1 à n

La somme des entiers de 1 à n vaut

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}.$$

Une anecdote célèbre raconte que le mathématicien allemand Carl Friedrich Gauss aurait découvert cette formule lorsqu'il était encore écolier.

Son professeur lui aurait demandé de calculer la somme des entiers de 1 à 100 afin d'occuper la classe pendant un moment. Mais Gauss remarqua rapidement que l'on pouvait associer les nombres par paires :

$$1 + 100 = 101, \quad 2 + 99 = 101, \quad 3 + 98 = 101, \quad \dots$$

Chaque paire donne la même somme, 101, et il y a 50 paires. La somme vaut donc $50 \times 101 = 5050$.

Ce raisonnement fonctionne pour tout n et conduit à la formule générale $\frac{n(n+1)}{2}$.

Cette formule permet notamment de connaître rapidement la somme totale des objets, ce qui sera utile pour déterminer la capacité du sac dans les problèmes de partition.

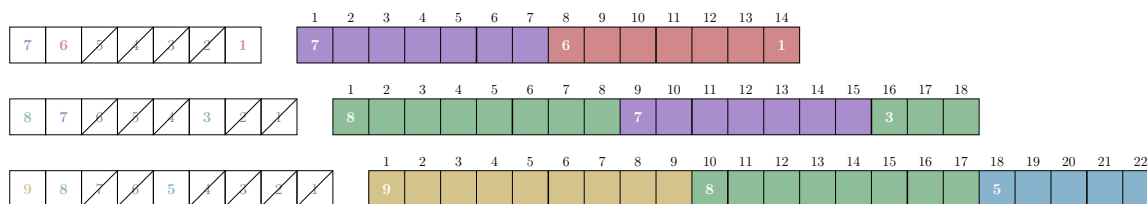


FIGURE 3 – Partition parfaite des entiers de 1 à n pour $n = 7, 8, 9$.

Définition 3.1 – Algorithme

Un *algorithme* est une suite d'étapes simples qui permet de résoudre un problème. Le nombre d'étapes peut varier en fonction de la taille des données.

Remarque 3.2

- Plusieurs algorithmes peuvent répondre à un même problème.
- Un algorithme peut répondre à plusieurs problèmes.

Exercice 3.2 – Algorithme pour partition parfaite (20 min)

Proposer un algorithme pour trouver une partition parfaite des entiers de 1 à n .

Epigramme d'Alan J. Perlis

Écrire un mauvais programme est facile ; comprendre un bon programme est difficile.

4 Algorithme glouton



Vous avez maintenant compris comment jongler avec les entiers et trouver des partitions parfaites... mais qui a envie de tester toutes les combinaisons possibles ? L'algorithme glouton correspond à la manière la plus intuitive de résoudre le problème pour les débutants. Il construit une solution étape par étape, sans se compliquer la vie. Ce n'est pas toujours la voie optimale, mais c'est simple, rapide et permet de se familiariser avec les algorithmes.

Jusqu'à présent, nous avons manipulé des sacs d'entiers sans leur donner de sens concret. Pour appliquer les algorithmes, nous allons maintenant considérer chaque entier comme la *taille* d'un objet réel. Ainsi, chaque entier du sac correspond à un objet que l'on peut placer dans un sac de capacité donnée. Dans les sections suivantes, nous utiliserons donc le terme *objet* tout en gardant à l'esprit qu'il sont seulement caractérisés par un entier, leur taille.

Définition 4.1 – Algorithme glouton

Un *algorithme glouton* est un algorithme qui, étape par étape, fait toujours le choix considéré comme le meilleur à ce moment précis (choix *localement optimal*). Dans certains problèmes, cette approche conduit à un optimum global. Dans le cas général, c'est une *heuristique* qui ne garantit pas de trouver l'optimum global.

Epigramme d'Alan J. Perlis

Si quelqu'un hoche la tête pendant que vous expliquez un algorithme, réveillez-le.

Algorithme 4.1 – Algorithme glouton

- Déterminer la capacité du sac.
- Trier les objets par ordre décroissant de taille.
- Tant qu’il reste des objets à placer :
 - placer le plus grand objet dans le sac si sa capacité le permet ;
 - Sinon, retirer l’objet ;
 - Si le sac est plein, arrêter l’algorithme.

Exercice 4.1 – Glouton avec type 1 (15 min)

Id	Taille	Entiers	Capacité
1	6	25, 22, 14, 12, 9, 4	43
2	6	25, 20, 18, 11, 4, 2	40
3	9	21, 17, 15, 13, 9, 6, 5, 4, 2	46
4	9	24, 18, 15, 14, 13, 7, 6, 2, 1	50
5	12	25, 23, 22, 20, 13, 11, 9, 7, 6, 5, 3, 2	73
6	15	25, 23, 22, 20, 19, 18, 16, 15, 12, 11, 8, 5, 3, 2, 1	100

TABLE 1 – Appliquer l’algorithme glouton sur une instance de type 1.

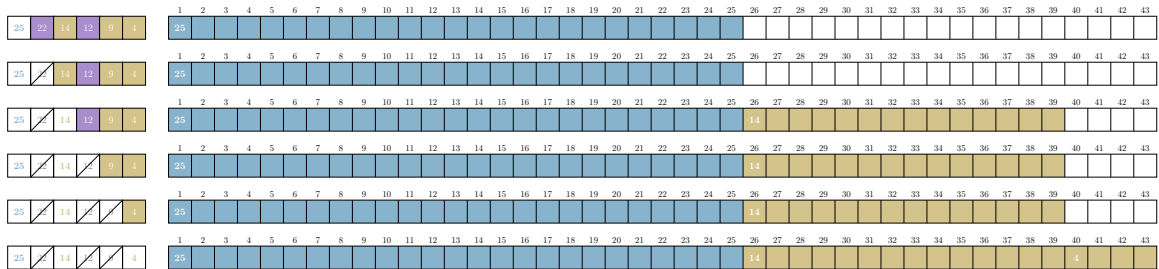


FIGURE 4 – Application de l’algorithme glouton sur l’instance #1.

Remarque 4.1 – Nombre d’étapes de l’algorithme glouton

Pour un ordre donné des objets, le glouton est très rapide :

- Chaque objet est examiné au plus une fois pour être placé dans le sac ;
- Le nombre d’étapes est donc proportionnel au nombre d’objets n .

En pratique, même pour plusieurs milliers d’objet le glouton trouve une partition presque sans transpirer.

Epigramme d'Alan J. Perlis

La complexité ne se communique pas, elle se pressent.

Exercice 4.2 – Glouton avec type 2 (15 min)

Id	Taille	Entiers	Capacité
7	6	22, 20, 15, 14, 9, 3	41
8	6	23, 20, 18, 17, 12, 10	50
9	9	22, 20, 17, 13, 9, 6, 5, 4, 2	49
10	9	24, 20, 15, 10, 8, 7, 6, 4, 1	47
11	12	25, 23, 18, 16, 15, 12, 11, 9, 7, 6, 4, 3	74
12	15	25, 23, 21, 19, 18, 17, 15, 14, 13, 10, 7, 6, 5, 4, 3	100

TABLE 2 – Appliquer l'algorithme glouton sur une instance de type 2.

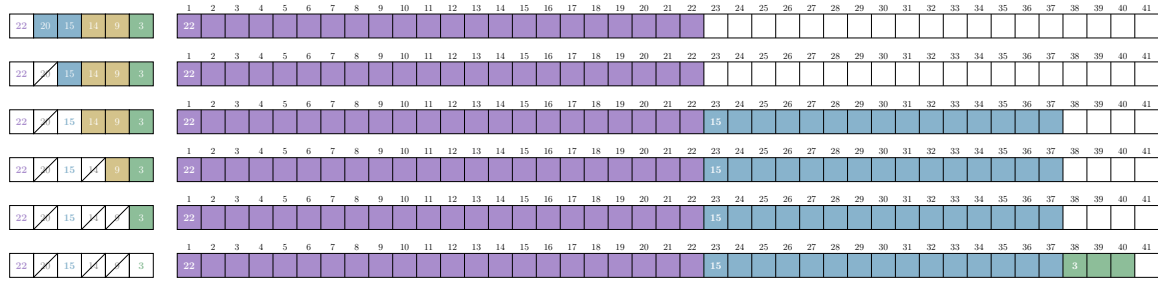


FIGURE 5 – Application de l'algorithme glouton sur l'instance #7.

Remarque 4.2 – À propos de l'algorithme glouton

- On peut obtenir d'autres solutions en modifiant l'ordre dans lequel les objets sont considérés.
- Des ordres différents donnent parfois la même solution.
- Répéter le glouton avec plusieurs ordres différents permet de générer plusieurs solutions.

Exercice 4.3 – Partition de 1 à n avec le glouton (10 min)

Appliquer l'algorithme glouton pour trouver une partition des entiers de 1 à $n = 15, 16$.

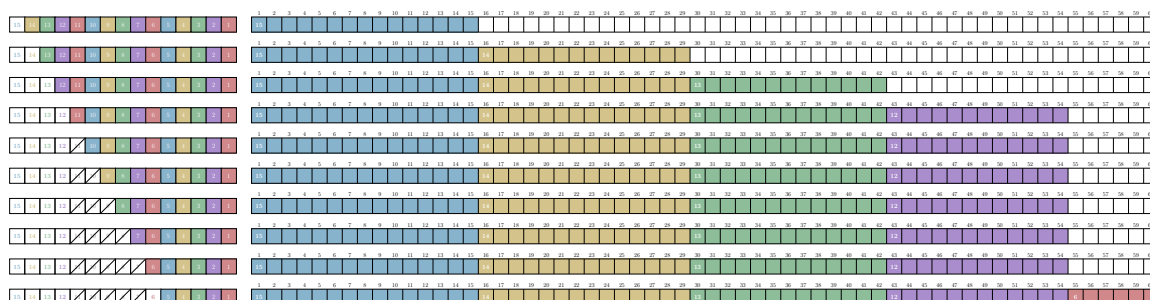


FIGURE 6 – Partition des entiers de 1 à 15.

Remarque 4.3 – Optimalité du glouton pour la partition des entiers de 1 à n

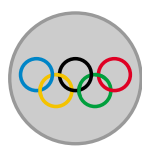
L'algorithme glouton est optimal pour le problème de partition des entiers de 1 à n :

- il existe toujours une partition parfaite des entiers de 1 à n ;
- l'algorithme glouton en trouve toujours une.

Preuve : supposons qu'à une étape de l'algorithme on ne puisse pas placer l'entier k dans le sac. Dans ce cas, la capacité restante du sac est égale à la différence entre la capacité totale et la somme des entiers déjà placés dans le sac.

Or les entiers encore disponibles sont exactement $1, 2, \dots, k$. Il reste donc toujours un entier égal à cette capacité restante, que l'on peut placer dans le sac. L'algorithme peut donc toujours compléter le sac.

5 Algorithme glouton répété



L'algorithme glouton semblait prometteur, mais il passe parfois juste à côté d'une partition parfaite. Et pourtant, en effectuant quelques échanges entre les objets, la solution finit par apparaître. Alan J. Perlis, vous glisserait volontiers qu'un programme devient souvent plus intéressant lorsqu'on le relance plusieurs fois.

L'idée est donc simple : si le glouton échoue, recommençons, mais en modifiant légèrement les conditions initiales. C'est précisément le principe de l'algorithme glouton répété : lancer le glouton plusieurs fois pour explorer d'autres possibilités en espérant être assez chanceux pour trouver une partition parfaite.

Epigramme d'Alan J. Perlis

Si l'on écrivait des programmes dès l'enfance, on saurait probablement mieux les lire une fois adulte.

Algorithme 5.1 – Algorithme glouton répété

Répéter tant que tous les objets ne sont pas dans le sac :

- Appliquer l’algorithme glouton.
- Si le sac est plein, arrêter l’algorithme.
- Sinon, retirer le plus grand objet du sac et recommencer.

Exercice 5.1 – Glouton répété avec type 2 (15 min)

Id	Taille	Entiers	Capacité
7	6	22, 20, 15, 14, 9, 3	41
8	6	23, 20, 18, 17, 12, 10	50
9	9	22, 20, 17, 13, 9, 6, 5, 4, 2	49
10	9	24, 20, 15, 10, 8, 7, 6, 4, 1	47
11	12	25, 23, 18, 16, 15, 12, 11, 9, 7, 6, 4, 3	74
12	15	25, 23, 21, 19, 18, 17, 15, 14, 13, 10, 7, 6, 5, 4, 3	100

TABLE 3 – Appliquer l’algorithme glouton répété sur une instance de type 2.



FIGURE 7 – Application de l’algorithme glouton répété sur l’instance #7.

Remarque 5.1 – À propos de l’algorithme glouton répété

L’algorithme glouton répété peut s’arrêter dès que tous les objets sont dans le sac, car retirer des objets ne permettrait pas d’améliorer la solution : le sac serait forcément moins rempli.

L’algorithme glouton répété examine uniquement des solutions distinctes : à chaque itération, il retire le plus grand objet restant avant de relancer le glouton. Or, cet objet est toujours inclus dans la solution construite par le glouton.

Remarque 5.2 – Nombre d’étapes de l’algorithme glouton répété

- L’algorithme explore au plus n solutions (une par objet successivement éliminé).
- L’algorithme glouton examine au plus les n objets.
- Le nombre total d’étapes de l’algorithme glouton est donc proportionnel à $n^2 = n \times n$.

Exercice 5.2 – Glouton répété avec type 3 (20 min)

Id	Taille	Entiers	Capacité
13	6	25, 20, 17, 14, 11, 8	47
14	6	23, 20, 17, 13, 12, 5	45
15	9	22, 19, 15, 14, 12, 10, 4, 3, 1	50
16	9	25, 19, 13, 12, 10, 8, 7, 3, 1	49
17	12	20, 17, 16, 14, 13, 12, 11, 10, 9, 6, 5, 3	68
18	15	25, 22, 20, 19, 18, 15, 14, 13, 11, 10, 9, 7, 6, 5, 2	98

TABLE 4 – Appliquer l’algorithme glouton répété sur une instance de type 3.

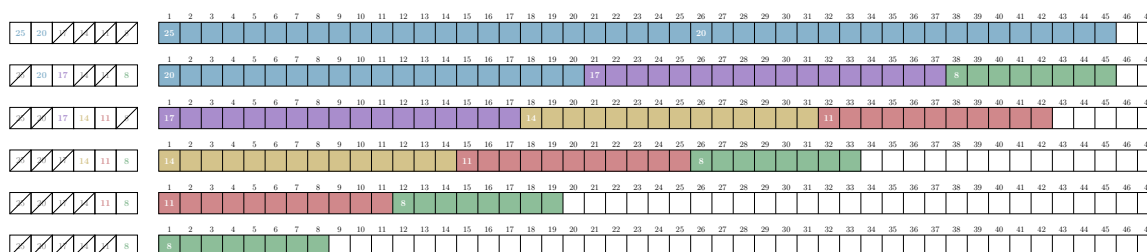


FIGURE 8 – Application de l’algorithme glouton répété sur l’instance #13.

6 Programmation dynamique



Cette fois, vous avez tout essayé. Le glouton répété n’a rien trouvé et vos tentatives d’échanges sont restées vaines. Malgré tous vos efforts la partition parfaite reste introuvable ou peut-être simplement bien cachée. Malheureusement, votre patron exige une réponse claire : la partition parfaite existe-t-elle ou non ?

C’est alors que Richard Bellman intervient avec une idée plus méthodique. Plutôt que d’explorer les possibilités un peu au hasard, il propose de construire progressivement toutes les sommes que l’on peut atteindre avec les objets disponibles. La méthode est plus coûteuse en temps, mais aussi plus difficile à imaginer et à mettre en œuvre. Il va falloir être rusé : la programmation dynamique est pour les champions de l’algorithmique.

Epigramme d’Alan J. Perlis

Pour comprendre un programme, il faut se mettre dans la peau de la machine et du programme.

Algorithme 6.1 – Algorithme de programmation dynamique

- Déterminer la capacité du sac.
- Trier les objets par ordre décroissant.
- Répéter tant qu'il reste des objets à placer :
 - Pour chaque case marquée, en partant de la dernière (à droite) :
 - Calculer la case atteinte en plaçant immédiatement l'objet après la case marquée.
 - Si cette case n'est pas encore marquée, placer un marqueur correspondant à l'objet.
 - Si la dernière case (capacité) est marquée, le sac est plein et l'algorithme s'arrête.

Nous savons désormais quelles sommes sont atteignables, ce qui est déjà une bonne nouvelle. En revanche, les solutions elles-mêmes restent cachées : à nous de les reconstruire en interprétant les marqueurs, avec l'algorithme comme guide.

Algorithme 6.2 – Reconstruction du sac en programmation dynamique

- Tant qu'il reste des marqueurs dans le sac :
- Sélectionner le marqueur le plus à droite.
 - Placer l'objet correspondant à ce marqueur dans le sac en retirant les marqueurs sous l'objet.

Exercice 6.1 – Programmation dynamique avec type 3 (20 min)

Id	Taille	Entiers	Capacité
13	6	25, 20, 17, 14, 11, 8	47
14	6	23, 20, 17, 13, 12, 5	45
15	9	22, 19, 15, 14, 12, 10, 4, 3, 1	50
16	9	25, 19, 13, 12, 10, 8, 7, 3, 1	49
17	12	20, 17, 16, 14, 13, 12, 11, 10, 9, 6, 5, 3	68
18	15	25, 22, 20, 19, 18, 15, 14, 13, 11, 10, 9, 7, 6, 5, 2	98

TABLE 5 – Appliquer la programmation dynamique sur une instance de type 3.

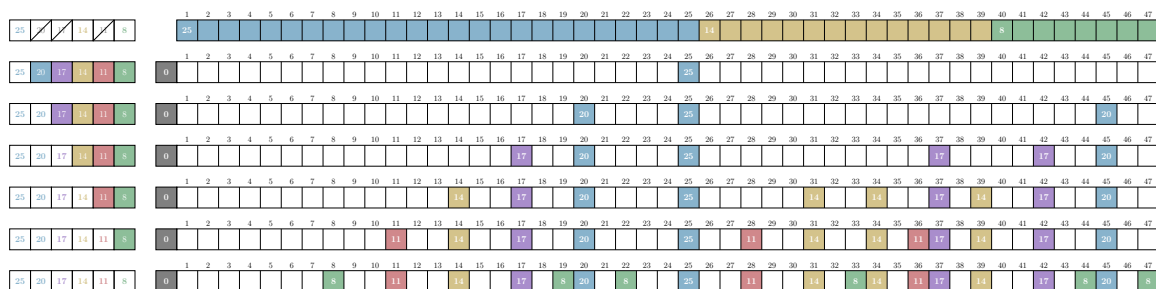


FIGURE 9 – Application de la programmation dynamique sur l'instance #13.

Remarque 6.1 – Nombre d'étapes de la programmation dynamique

L'algorithme de programmation dynamique est beaucoup plus gourmand en étapes que les gloutons. Pour n objets et une capacité C du sac :

- Il faut examiner toutes les cases marquées pour chaque objet, et il y a au plus C cases marquées ;
- Le nombre d'étapes est donc proportionnel à $n \cdot C$, ce qui est bien plus que le glouton.

Cette complexité ne dépend pas seulement de la taille de l'entrée (le nombre d'objets), mais aussi de leurs valeurs (ici la capacité du sac). Pour une taille et des valeurs raisonnables, l'algorithme reste rapide, mais pour de grandes tailles et valeurs, le temps de calcul peut devenir important.

Exercice 6.2 – Programmation dynamique avec type 4 (25 min)

Id	Taille	Entiers	Capacité
19	6	24, 21, 19, 15, 9, 4	46
20	6	25, 22, 15, 13, 9, 1	42
21	9	25, 23, 18, 15, 13, 10, 8, 5, 3	60
22	9	25, 24, 15, 14, 13, 12, 11, 2, 1	58
23	10	24, 22, 20, 18, 16, 14, 12, 10, 6, 4	73
24	11	24, 22, 20, 18, 16, 14, 12, 10, 8, 6, 4	77

TABLE 6 – Appliquer la programmation dynamique sur une instance de type 4.

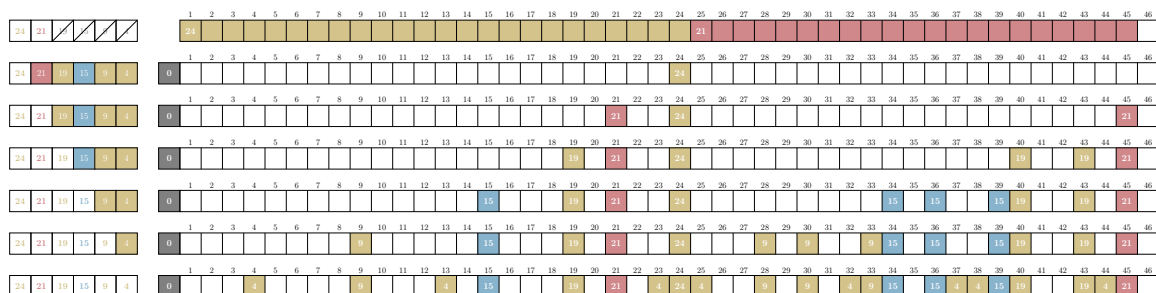


FIGURE 10 – Application de la programmation dynamique sur l'instance #19.

Remarque 6.2 – Optimalité de la programmation dynamique

L'algorithme de programmation dynamique garantit de trouver la partition la plus équilibrée :

- il trouve une partition parfaite si elle existe ;
- sinon, il minimise la différence entre les sommes des deux sous-sacs.

On sacrifie la rapidité pour être sûr d'obtenir la solution optimale.

Remarque 6.3 – Une ouverture en profondeur de Bellman

Il existe une méthode exacte plus générale pour résoudre ce type de problème : *Tester-et-Générer*. Simple à comprendre et quelquefois facile à programmer, elle devient vite très pénible à exécuter à la main, tant elle explore toutes les possibilités par sauts de puce. Après avoir déjà transpiré sur la programmation dynamique, notre patience a des limites... et nous n'irons pas plus loin dans cette voie !

Remarque 6.4 – Bellman botte en touche !

Il existe aussi une pléthore de méthodes approximatives plus élaborées pour s'échapper des limites des algorithmes gloutons. Elles reposent souvent sur des échanges d'objets et s'inspirent librement de phénomènes naturels : colonies de fourmis, essaims d'abeilles, etc. Mais Richard Bellman rechigne à vous en dire davantage : selon lui, même efficaces, ces approches n'ont pas tout à fait la même noblesse ni la même rigueur que les méthodes exactes. Vous devrez donc les découvrir sans son aide !

7 Citius, altius, fortius (plus vite, plus haut, plus fort)

Votre esprit est maintenant limpide ! Vous voilà initié aux arcanes des partitions. Mais une question demeure : comment ce labeur acharné peut-il se transformer en véritable flemme olympique ?

C'est alors qu'Alan, malicieusement, vous souffle à nouveau à l'oreille :

Epigramme d'Alan J. Perlis

On croit comprendre en apprenant, on est plus confiant en écrivant, encore plus en enseignant... mais on en est vraiment sûr en programmant.

Il ne vous reste donc qu'une ultime épreuve.

Exercice 7.1 – Olympiade des partitions (? min)

Écrire un programme capable de résoudre le problème de partition et le soumettre au juge olympique.

À ce prix seulement, vous pourrez espérer rejoindre le panthéon des partitions. Et qui sait ? Peut-être pourrez-vous enfin profiter des Jeux... autrement que derrière un écran.

Epigramme d'Alan J. Perlis

Pensez à toute l'énergie mentale dépensée pour tenter de tracer la frontière entre « algorithme » et « programme ».

8 Problèmes connexes

Le problème de partition appartient à une grande famille de problèmes d'optimisation étudiés en informatique et en recherche opérationnelle. De nombreux problèmes célèbres lui sont étroitement liés.

Epigramme d'Alan J. Perlis

Tout programme fait au moins deux choses : ce pour quoi il a été conçu ... et une autre pour laquelle il ne l'a pas été.

Définition 8.1 – Problème de la somme de sous-ensembles

On dispose d'un sac d'entiers et d'une capacité donnée. La question est de savoir s'il existe un sous-ensemble d'objets dont la somme est exactement égale à cette capacité. Le problème de partition en est un cas particulier où la capacité est la moitié de la somme totale des entiers.

Définition 8.2 – Problème d’ordonnancement sur deux machines

Un problème très proche apparaît en *ordonnancement*. Supposons que plusieurs tâches doivent être réparties sur deux machines identiques. Chaque tâche a une durée d’exécution. On cherche à minimiser le délai total, c’est-à-dire la date de fin de la dernière tâche. Pour cela, il faut répartir les tâches entre les deux machines de manière à équilibrer au mieux leurs durées totales.

Ce problème revient donc à chercher une partition des durées des tâches aussi équilibrée que possible.

Remarque 8.1 – Un algorithme peut répondre à plusieurs problème !

Tous les algorithmes introduits fonctionnent pour la somme des sous-ensembles et l’ordonnancement sur deux machines.

Définition 8.3 – Problème du sac à dos

Chaque objet possède une taille et une valeur. Le but est de choisir des objets à placer dans un sac de capacité limitée de manière à maximiser la valeur totale.

Contrairement au problème de partition, il ne s’agit plus de trouver une somme exacte, mais de sélectionner la meilleure combinaison possible en termes de valeurs.

Définition 8.4 – Problème de mise en boîtes

On dispose de plusieurs sacs de capacité fixe et d’objets de tailles différentes. Le but est de ranger tous les objets en utilisant le moins de sacs possible.

Ce problème généralise l’idée de remplir un sac avec des objets de tailles données.

Ces problèmes [1–5] apparaissent dans de nombreux domaines : logistique, planification de tâches, allocation de ressources ou encore gestion de données. Ils illustrent l’importance des algorithmes pour organiser et optimiser des ressources limitées.

Epigramme d’Alan J. Perlis

La simplicité ne précède pas la complexité, elle la suit.

A Guide pour les animateurs

Cette suite d’activités a été conçue pour être adaptée à différents publics, du primaire au supérieur. Le scénario dépendra aussi de la familiarité et l’intérêt de l’animateur.

TODO

B Tableau des instances

Id	Taille	Entiers	Capacité	Type
1	6	25, 22, 14, 12, 9, 4	43	1
2	6	25, 20, 18, 11, 4, 2	40	1
3	9	21, 17, 15, 13, 9, 6, 5, 4, 2	46	1
4	9	24, 18, 15, 14, 13, 7, 6, 2, 1	50	1
5	12	25, 23, 22, 20, 13, 11, 9, 7, 6, 5, 3, 2	73	1
6	15	25, 23, 22, 20, 19, 18, 16, 15, 12, 11, 8, 5, 3, 2, 1	100	1
7	6	22, 20, 15, 14, 9, 3	41	2
8	6	23, 20, 18, 17, 12, 10	50	2
9	9	22, 20, 17, 13, 9, 6, 5, 4, 2	49	2
10	9	24, 20, 15, 10, 8, 7, 6, 4, 1	47	2
11	12	25, 23, 18, 16, 15, 12, 11, 9, 7, 6, 4, 3	74	2
12	15	25, 23, 21, 19, 18, 17, 15, 14, 13, 10, 7, 6, 5, 4, 3	100	2
13	6	25, 20, 17, 14, 11, 8	47	3
14	6	23, 20, 17, 13, 12, 5	45	3
15	9	22, 19, 15, 14, 12, 10, 4, 3, 1	50	3
16	9	25, 19, 13, 12, 10, 8, 7, 3, 1	49	3
17	12	20, 17, 16, 14, 13, 12, 11, 10, 9, 6, 5, 3	68	3
18	15	25, 22, 20, 19, 18, 15, 14, 13, 11, 10, 9, 7, 6, 5, 2	98	3
19	6	24, 21, 19, 15, 9, 4	46	4
20	6	25, 22, 15, 13, 9, 1	42	4
21	9	25, 23, 18, 15, 13, 10, 8, 5, 3	60	4
22	9	25, 24, 15, 14, 13, 12, 11, 2, 1	58	4
23	10	24, 22, 20, 18, 16, 14, 12, 10, 6, 4	73	4
24	11	24, 22, 20, 18, 16, 14, 12, 10, 8, 6, 4	77	4

C Liste des exercices

2.1	Partition de quatre entiers consécutifs (5 min)	4
2.2	Partition de six entiers consécutifs (5 min)	5
2.3	Échanges partiels (5 min)	6
3.1	Partition parfaite de 1 à n (10 min)	7
3.2	Algorithme pour partition parfaite (20 min)	8
4.1	Glouton avec type 1 (15 min)	9
4.2	Glouton avec type 2 (15 min)	10
4.3	Partition de 1 à n avec le glouton (10 min)	10
5.1	Glouton répété avec type 2 (15 min)	12
5.2	Glouton répété avec type 3 (20 min)	13
6.1	Programmation dynamique avec type 3 (20 min)	14

6.2	Programmation dynamique avec type 4 (25 min)	15
7.1	Olympiade des partitions (? min)	17

D Licence et reproductibilité

Cette activité est libre, ouverte, et reproductible dans le respect des licences. Ainsi, toute personne peut vérifier, reproduire et prolonger les résultats proposés.

<https://github.com/arnaud-m/flemolymp>

Elle contient notamment :

- le code source et les données sous la forme d’une extension R, distribué sous licence MIT ;
- le contenu pédagogique (textes, images, schémas, diagrammes) distribué sous licence CC BY 4.0.
- le source L^AT_EX de tous les documents (manuel, fiche technique, solutions) ;
- les fichiers nécessaires à la fabrication des éléments physiques via découpeuse laser ou imprimante 3D ;
- les illustrations et images utilisées dans l’activité, libres de droits ou sous licence compatible ;
- les instructions détaillées pour la mise en place de l’activité, incluant la préparation du matériel et des variantes selon le niveau des participants.

Références

- [1] Problème de mise en boîtes (bin packing problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_bin_packing.
- [2] Problème de sac-à-dos (knapsack problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_du_sac_%C3%A0_dos.
- [3] Problème d’ordonnancement avec des machines identiques (identical-machines scheduling). https://en.wikipedia.org/wiki/Identical-machines_scheduling.
- [4] Problème de partition (partition problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_partition.
- [5] Problème de la somme des sous-ensembles (subset sum problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_la_somme_de_sous-ensembles.
- [6] maix. Images de médailles de bronze, argent, ou or disponibles sur wikimedia commons. <https://commons.wikimedia.org/w/index.php?curid=1759963>, 2007. CC BY-SA 2.5.
- [7] Sylvano Martello and Paolo Toth. *Knapsack Problems*. Wiley, 1990.
- [8] Stephan Mertens. The easiest hard problem : Number partitioning, 2003. URL <https://arxiv.org/abs/cond-mat/0310317>.

- [9] Alan J. Perlis. Special feature : Epigrams on programming. *SIGPLAN Not.*, 17(9) :7–13, September 1982. ISSN 0362-1340. doi : 10.1145/947955.1083808. URL <https://doi.org/10.1145/947955.1083808>.